Django-তে MongoDB কানেক্ট করা একটু আলাদা, কারণ Django ডিফল্টভাবে SQL ডাটাবেস (যেমন PostgreSQL বা MySQL) এর জন্য তৈরি। তবে পাইথনে এটি করার দুটি জনপ্রিয় উপায় আছে।

সবচেয়ে সহজ এবং কার্যকর উপায় হলো **Djongo** ব্যবহার করা। এটি আপনাকে Django-র চিরাচরিত `Models` এবং `Admin Panel` ব্যবহার করতে দেয়, কিন্তু ডাটা সেভ করে MongoDB-তে।

ধাপ ১: প্রয়োজনীয় লাইব্রেরি ইনস্টল করুন

আপনার টার্মিনালে নিচের কমান্ডটি দিন:

Bash

```
pip install djongo
```

(দ্রষ্টব্য: আপনার পিসিতে MongoDB ইনস্টল করা থাকতে হবে অথবা MongoDB Atlas-এর ক্লাউড ইউআরএল থাকতে হবে।)

ধাপ ২: Settings.py পরিবর্তন করুন

আপনার Django প্রজেক্টের `settings.py` ফাইলে গিয়ে `DATABASES` সেকশনটি এভাবে আপডেট করুন:

Python

```
DATABASES = {
    'default': {
        'ENGINE': 'djongo',
        'NAME': 'your_db_name',  # ডাটাবেসের নাম
        'ENFORCE_SCHEMA': False,
        'CLIENT': {
            'host': 'mongodb://localhost:27017/', # অথবা Atlas এর লিঙ্ক
        }
    }
}
```

ধাপ ৩: মডেল তৈরি করা

এখন আপনি আগের মতোই সাধারণ Django মডেল লিখতে পারবেন। MongoDB ব্যাকএন্ডে এটি অটোমেটিক হ্যান্ডেল করবে।

Python

```
from django.db import models

class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.FloatField()
    metadata = models.JSONField() # MongoDB এর সুবিধা হলো আপনি সরাসরি JSON
সেভ করতে পারেন

    def __str__(self):
        return self.name
```

ধাপ ৪: মাইগ্রেশন করা

Bash

```
python manage.py makemigrations
python manage.py migrate
```

অল্টারনেটিভ উপায়: PyMongo (Direct Connection)

যদি আপনি Django-র মডেলে (ORM) অভ্যস্ত না হন এবং সরাসরি MongoDB কোড লিখতে চান (যেমন আপনি মাইক্রোসার্ভিসে করেছিলেন), তবে **PyMongo** ব্যবহার করতে পারেন।

উদাহরণ:

Python

```
import pymongo

client = pymongo.MongoClient("mongodb://localhost:27017/")
db = client["test_database"]
collection = db["test_collection"]

# ডাটা ইনসার্ট করা
collection.insert_one({"name": "Abir", "status": "Learning MongoDB"})
```

আপনি কোনটি পছন্দ করবেন?

1. **Djongo:** যদি আপনি Django-র Admin Panel এবং Form ব্যবহার করতে চান।
2. **PyMongo:** যদি আপনি একদম Raw MongoDB কুয়েরি চালিয়ে দ্রুত কাজ করতে চান (যা মাইক্রোসার্ভিসে বেশি জনপ্রিয়)।

আপনি কি চান আমি আপনার "Stay-Bangla" প্রজেক্টের জন্য একটি ডেমো MongoDB কানেকশন কোড লিখে দেই?

Django-তে ORM (Object-Relational Mapper) ব্যবহার করে MongoDB কানেক্ট করাটা দারুণ বুদ্ধিমানের কাজ। এতে আপনার কোড অনেক ক্লিন থাকে এবং আপনাকে RAW কুয়েরি লিখতে হয় না।

আমরা **Djongo** ব্যবহার করে এটি করব। নিচে আপনার **Stay-Bangla** প্রোজেক্টের ধাঁচে একটি কমপ্লিট সেটআপ দেওয়া হলো:

---

১. মডেল তৈরি করা (models.py)

Django ORM-এর মজা হলো আপনি পাইথন ক্লাসের মাধ্যমেই ডাটাবেস টেবিল (বা কালেকশন) ডিফাইন করতে পারেন।

Python

```
from django.db import models

class Room(models.Model):
    title = models.CharField(max_length=200)
    description = models.TextField()
    price_per_night = models.DecimalField(max_digits=10, decimal_places=2)
    is_available = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.title
```

২. ডাটাবেসে ডাটা সেভ করা (ORM ব্যবহার করে)

এখন আপনাকে আর SQL বা MongoDB কুয়েরি লিখতে হবে না। আপনি শুধু পাইথন অবজেক্ট তৈরি করবেন।

Python

```
# নতুন একটি রুম অ্যাড করা
new_room = Room(
    title="Luxury Sea View Room",
    description="Beautiful view of Cox's Bazar",
    price_per_night=5000.00
)
new_room.save() # এটি অটোমেটিক MongoDB-তে ইনসার্ট হবে
```

৩. ডাটা রিড করা (QuerySet)

আপনার ওয়েবসাইট বা এপিআই-তে ডাটা দেখানোর জন্য নিচের মতো করে কুয়েরি করবেন:

Python

```
# সব রুমের লিস্ট আনা
all_rooms = Room.objects.all()

# নির্দিষ্ট একটি রুম খুঁজে বের করা
single_room = Room.objects.get(id=1)

# ৫০০০ টাকার নিচের রুমগুলো ফিল্টার করা
cheap_rooms = Room.objects.filter(price_per_night__lt=5000)
```

কেন MongoDB-তে Django ORM (Djongo) ব্যবহার করবেন?

- **Admin Panel:** আপনি Django-র সেই চমৎকার অ্যাডমিন প্যানেলটি ব্যবহার করতে পারবেন যা আপনি SQL-এ করতেন।
- **Security:** ORM ব্যবহার করলে আপনার অ্যাপ **SQL Injection** এর মতো অ্যাটাক থেকে নিরাপদ থাকে।
- **Easy Migrations:** ডাটাবেসের স্ট্রাকচার চেঞ্জ করলে `makemigrations` দিয়ে সহজেই হ্যান্ডেল করা যায়।

একটি ছোট টিপস:

আপনার `settings.py`-তে যখন `DATABASES` কনফিগার করবেন, তখন নিশ্চিত করুন যে `ENGINE` হিসেবে `'djongo'` দেওয়া আছে। কারণ ডিফল্ট `'django.db.backends.sqlite3'` দিলে এটি কিন্তু MongoDB-তে ডাটা পাঠাবে না।

আপনি কি আপনার প্রোজেক্টে এই সেটআপটি ট্রাই করছেন? কোনো এরর (Error) আসলে আমাকে কোডটি দিতে পারেন, আমি ফিক্স করে দেব।

আসলে এখানে একটি ছোট ভুল বোঝাবুঝি হচ্ছে। আপনি যেহেতু **Django** (Python) ব্যবহার করছেন, তাই আপনার `npm install` ব্যবহার করার প্রয়োজন নেই।

পার্থক্যটা বুঝে নিন:

- **npm (Node Package Manager):** এটি ব্যবহার করা হয় যখন আপনি **JavaScript** বা **Node.js** নিয়ে কাজ করেন।

- **pip (Python Package Index):** এটি ব্যবহার করা হয় যখন আপনি **Python** বা **Django** নিয়ে কাজ করেন।

যেহেতু আপনার প্রজেক্টটি পাইথনে, তাই আপনাকে `pip` ব্যবহার করতে হবে।

আপনার যা যা ইনস্টল করতে হবে:

টার্মিনালে নিচের এই কমান্ডগুলো একে একে দিন:

১. **Djongo ইনস্টল করুন (এটি Django ORM কে MongoDB এর সাথে কানেক্ট করে):**

Bash

```
pip install djongo
```

২. **dnspython ইনস্টল করুন (এটি MongoDB Atlas বা ক্লাউড কানেকশনের জন্য লাগে):**

Bash

```
pip install dnspython
```

কেন `npm` নয়?

আপনি যদি আপনার প্রজেক্টের ফ্রন্ট-এন্ডে **React** বা **Vue** ব্যবহার করতেন, তবে সেখানে প্যাকেজ ম্যানেজ করার জন্য `npm` লাগত। কিন্তু ব্যাক-এন্ডে ডাটাবেস কানেকশনের জন্য পাইথনের লাইব্রেরিই যথেষ্ট।

পরবর্তী ধাপ:

এগুলো ইনস্টল করার পর আপনার `settings.py` ফাইলে গিয়ে ডাটাবেস কনফিগারেশনটা আপডেট করে নিন। আপনি কি আপনার `settings.py` এর ডাটাবেস অংশটুকু আমাকে দেখাবেন? তাহলে আমি আপনার প্রজেক্টের নাম অনুযায়ী সেটি লিখে দিতে পারব।

আপনার পিসিতে কি MongoDB কম্পাস (Compass) বা লোকাল সার্ভার ইনস্টল করা আছে? না থাকলে আমি সেটিও ডাউনলোড করার লিঙ্ক দিয়ে দিচ্ছি।অসাধারণ! কানেকশন স্ট্রিং পেয়ে গেছেন মানে আপনার অর্ধেক কাজ শেষ।

এখন শুধু আপনার Django প্রোজেক্টের সাথে এটিকে যুক্ত করার পালা। আমি নিচে আপনার জন্য একদম রেডিমেড কোড লিখে দিচ্ছি যা আপনি সরাসরি `settings.py`-তে বসাতে পারবেন।

---

১. `settings.py` আপডেট করুন

আপনার `DATABASES` সেকশনটি মুছে দিয়ে নিচের কোডটি বসান:

```
DATABASES = {
    'default': {
        'ENGINE': 'djongo',
        'NAME': 'stay_bangla_db',  # এখানে আপনার পছন্দমতো ডাটাবেস নাম দিন
        'ENFORCE_SCHEMA': False,
        'CLIENT': {
            'host': 'আপনার_কানেকশন_স্ট্রিং_এখানে_দিন', # Atlas থেকে পাওয়া লিঙ্কটি
এখানে বসান
        }
    }
}
```

> **সতর্কতা:** কানেকশন স্ট্রিং-এ যেখানে `<password>` আছে, সেখানে আপনার সেট করা পাসওয়ার্ডটি দিতে ভুলবেন না।

২. লাইব্রেরিগুলো একবার চেক করে নিন

কানেক্ট করার আগে নিশ্চিত হয়ে নিন যে এই তিনটি জিনিস ইনস্টল করা আছে (আমরা আগে আলাপ করেছি):

```
pip install djongo
pip install dnspython
pip install pymongo==3.12.3
```

৩. প্রথমবার রান করা

এখন নিচের কমান্ড দুটি দিন। যদি সব ঠিক থাকে, তবে আপনার MongoDB Atlas-এ অটোমেটিক সব টেবিল তৈরি হয়ে যাবে:

```
python manage.py makemigrations
python manage.py migrate
```

এর পর কী করবেন?

ডাটাবেস কানেক্ট হয়ে গেলে আপনি আগের মতোই `python manage.py createsuperuser` দিয়ে একটি অ্যাডমিন অ্যাকাউন্ট খুলে দেখতে পারেন যে কাজ করছে কি না।

আপনার কানেকশন কি সাকসেসফুল হয়েছে? কোনো লাল রঙের এরর (Error) দেখাচ্ছে কি? দেখালে আমাকে মেসেজ দিন, আমি আছি আপনার সাথে।

কোডিং নিয়ে কাজ করতে করতে আপনার মনটা এখন কি একটু হালকা লাগছে? আশা করি এই নতুন কিছু শেখা আপনার সময়টাকে ভালো করে দিচ্ছে।

হ্যাঁ, অবশ্যই পারবেন! জ্যাঙ্গোতে (Django) একই সাথে একাধিক ডাটাবেস (যেমন: একপাশে **PostgreSQL** এবং অন্যপাশে **MongoDB**) ব্যবহার করা খুব কমন একটি বিষয়। একে বলা হয় **Multiple Databases Support**।

এটি করার জন্য আপনাকে মূলত ৩টি ধাপ ফলো করতে হবে:

---

১. `settings.py` -তে ডাটাবেসগুলো ডিফাইন করা

আপনার `DATABASES` ডিকশনারিতে এখন একটির বদলে দুটি (বা তার বেশি) এন্ট্রি থাকবে।

```
DATABASES = {
    'default': {  # এটি আপনার মেইন ডাটাবেস (ধরি SQL)
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'main_db',
        'USER': 'admin',
        # ... অন্যান্য সেটিংস
    },
    'mongodb': {  # এটি আপনার সেকেন্ডারি ডাটাবেস
        'ENGINE': 'djongo',
        'NAME': 'stay_bangla_mongo',
        'CLIENT': {
            'host': 'your_mongodb_connection_string',
        }
    }
}
```

---

২. ডাটাবেস রাউটার (Database Router) তৈরি করা

জ্যাঙ্গোকে বলে দিতে হবে কোন মডেলের ডাটা কোন ডাটাবেসে যাবে। এজন্য একটি ছোট পাইথন ফাইল (যেমন: `db_routers.py`) বানাতে হবে।

```
class MyRouter:
    def db_for_read(self, model, **hints):
        if model._meta.app_label == 'my_mongo_app':
            return 'mongodb'
        return 'default'
```

```
    def db_for_write(self, model, **hints):
        if model._meta.app_label == 'my_mongo_app':
            return 'mongodb'
        return 'default'
```

এরপর `settings.py` -তে এটি যুক্ত করে দিন:
`DATABASE_ROUTERS = ['your_project.db_routers.MyRouter']`

---

৩. কোডিং করার সময় নির্দিষ্ট ডাটাবেস ব্যবহার করা

আপনি চাইলে কোড লেখার সময়ও বলে দিতে পারেন কোন ডাটাবেস থেকে ডাটা আসবে:

```
# 'mongodb' ডাটাবেস থেকে ডাটা আনা
rooms = Room.objects.using('mongodb').all()

# 'default' (SQL) ডাটাবেসে ডাটা সেভ করা
user.save(using='default')
```

কেন আপনি একাধিক ডাটাবেস ব্যবহার করবেন?

- **পারফরম্যান্স:** ইউজারের প্রোফাইল বা পেমেন্ট রাখুন SQL-এ (নিরাপদ), আর হোটেলের রিভিউ বা বড় বড় ডাটা রাখুন MongoDB-তে (দ্রুত)।
- **মাইক্রোসার্ভিস স্টাইল:** আপনার প্রোজেক্ট বড় হলে বিভিন্ন ফিচারের জন্য আলাদা আলাদা ডাটাবেস থাকলে সেটি ম্যানেজ করা সহজ হয়।

আপনার জন্য একটি ছোট প্রশ্ন:

আপনি কি আপনার **Stay-Bangla** প্রোজেক্টের **User Authentication** এবং **Booking**-এর জন্য আলাদা আলাদা ডাটাবেস রাখতে চাচ্ছেন? নাকি কোনো নির্দিষ্ট ফিচারের জন্য (যেমন: চ্যাট বা লগ সিস্টেম) আলাদা ডাটাবেস কথা ভাবছেন?

বললে আমি সেই অনুযায়ী কোডটি আরও সহজ করে বুঝিয়ে দিতে পারব। আপনার এই যে শেখার আগ্রহ—এটি কিন্তু প্রোগ্রামিং ক্যারিয়ারে অনেক দূর নিয়ে যাবে!